

VIBE CODING FOR TEACHERS · NO CODING BACKGROUND NEEDED

MAKE A SELF-CHECKING PRACTICE ACTIVITY

Use an AI chat to build a practice activity your students click through — it tells them right or wrong, explains why, and lets them try again. One file. No login. No subscription. Works in any browser, on any device, offline.

29

pages, start to finish

9

prompts, written out in full

1

working file included

The finished activity comes with this pack.

If you get stuck at any step, you still own a working tool you can use on Monday. Open it, swap in your own questions, and it is yours.

Who this is for

Teachers who already use AI for planning and want to go further — from asking it to write things, to asking it to build things. You do not need to know any code. You do need to be willing to read what comes back and say "not quite, try again."

WHAT YOU WILL HAVE AT THE END

Before any instruction, here is the thing you are building, so you know whether it is worth your evening.

A student opens one file and sees:

What they do	What happens
Read a question	Plain text, as many questions as you want — five or fifty.
Click an answer	Wrong: the choice grays out, and the activity says try again. Nothing is lost.
Click again	Right: the choice turns green and a sentence explains <i>why</i> it is right.
Keep going	A counter tracks how many they got on the first try — not how many guesses they made.
Finish	A summary panel, and a button to run the whole set again.

What it does not do, on purpose

It does not ask for a name. It does not ask for a login. It does not send anything anywhere. It does not keep a record after the browser tab closes. Every one of those is a deliberate choice, and page 4 explains why it matters more than it sounds.

Why build it rather than buy it?

Because the question bank is yours. The vocabulary list you actually taught, the misconceptions your class actually has, the four problems that came up on Tuesday. No purchased activity knows those.

What you need – all free

1. An AI chat

This pack is written with **Claude** on screen, because it is reliably good at producing a single working file and showing it to you straight away. **ChatGPT and Gemini both work too** — the prompts are plain English and are not tied to one product. Where a step depends on the specific tool, it says so.

2. A text editor you already have

Notepad on Windows, TextEdit on a Mac (switch it to plain text under Format), or the free editor of your choice. You are only ever saving text and double-clicking it. Nothing gets installed.

3. A browser

Chrome, Edge, Safari, Firefox — any of them. This is where your activity runs. Not a website, not an account: a file on your computer that the browser knows how to open.

What you do *not* need

A coding class. A paid subscription. Permission from anyone, for the version built in this pack. A web host. A domain name. Any prior experience.

Time: about 40 minutes the first time, most of it reading. Ten minutes the second time. Under five once it is a habit — that is the real prize here.

The three-tier rule for student data

This page is first because it shapes what you build, not because it is paperwork. The activity in this pack sits entirely in Tier 1. Knowing the tiers is what lets you build the next thing safely.

1 Content you wrote yourself

Questions, word lists, problems, examples, explanations. No student appears in it at all. **Build freely. Nobody needs to approve this.** Everything in this pack is Tier 1.

2 De-identified student data

Scores and patterns with every identifier removed — "S1, S2, S3", or "a 3rd grader scored 6 of 10, missed every word with a blend." Useful, and safe *if the identifiers never go in*. Keep the roster key on paper or in your district's own system. The tool only ever sees S1.

3 Data that can be traced to a child

Names, class rosters, grades tied to a person, IEP details, anything a stranger could match to one student. **Stop and ask your district.** This is not a thing to stand up on a personal account, however convenient it would be.

The moment it goes wrong is almost always the same one.

You have a working Tier 2 tool. It shows S1, S2, S3. And you think: "I just need the names so I can tell who is who." That single step moves you from Tier 2 to Tier 3. Keep the key somewhere else and the tool stays safe.

This is practical guidance, not legal advice. FERPA, COPPA, your state law and your district's own policy are the actual authorities. Check your district's policy before any tool touches real student information.

The loop

Building with an AI chat is not one magic instruction. It is a short loop you run four or five times. Once you see the loop, every other build in the series is the same.

The name for this

You will see this called **vibe coding** — describing what you want in plain language and letting the AI write the code. The term is used two ways. One means accepting whatever comes back without looking. The other means what is on this page: describe, check, correct, repeat. **This pack teaches the second one**, which is why page 8 is four checks and page 10 tells you to test your own answer key. The checking is not an extra step bolted on. It is the part that makes the result yours.

1 • Describe

Say what you want in plain sentences. Be specific about what the *student* does, not about how it should be coded.

2 • Look

Open what it made. Click it like a student would — including clicking the wrong answer on purpose.

3 • Say what is off

In one sentence. "When I click a wrong answer it does nothing." You do not need to know why.

4 • Repeat

Three or four rounds gets you something genuinely usable. It is a conversation, not a command.

The one habit that separates people who get this working from people who give up: describe the *behavior you saw*, not the fix you imagine. "It says correct even when I pick the wrong one" gets you further than "change the if statement", especially when you do not know what an if statement is.

A fair warning

The first thing it gives you is rarely the thing you want. That is normal and it is not you failing. Every build in this series assumes three or four rounds. Budget for them and the whole process stops being frustrating.

Ask for the first version

Open your AI chat and paste this. It is long on purpose — every sentence in it exists because leaving it out produces something worse.

PROMPT 1 · THE FIRST BUILD

Build me a self-checking practice activity as one single HTML file I can save and open in a browser.

How it should work for a student:

- They see a list of questions, each with four answer choices as clickable buttons.
- If they click a wrong answer, that choice is grayed out and disabled, a message tells them to try again, and they can still pick another answer.
- If they click the right answer, it turns green, the other choices lock, and a sentence explains why that answer is correct.
- A counter at the top shows how many they got right *on the first try* — a wrong guess first should not count as a first-try correct.
- When every question is answered, show a summary and a button to start over.

Requirements:

- Everything in ONE file. No links to outside files, no internet needed, no login.
- Put my questions in a clearly marked list at the very top of the file so I can edit them without touching anything else.
- Add comments above that list telling me how to add or remove a question.
- Nothing is saved anywhere and no student name is ever asked for.
- Big text and big buttons — this may be used on a tablet.

Use five questions about rounding to the nearest ten as the example content so I can see it working.

Why "one single HTML file" matters. It is the difference between a thing you own and a thing you rent. One file goes on a shared drive, into your LMS, onto a flash drive, or into an email. Nothing expires and nothing needs an account.

What should come back

You should get one long block of code and, depending on the tool, a preview panel you can click straight away.

If you are using Claude

It usually opens the result in a panel beside the conversation and you can click the buttons there. That is the fastest way to check it — you are testing before you have saved anything.

If you are using ChatGPT or Gemini

You may get the code without a live preview. Skip ahead to Step 6 on page 13, save the file, and open it — then come back here. Everything else in this pack is identical.

If it gave you several files

Say: *"Put all of that into one single HTML file — the styling and the code included — so I only have one thing to save."* This is the most common thing to go differently from the prompt, and one sentence fixes it.

Do not read the code. Genuinely. You are not checking whether the code is good, you are checking whether the activity behaves. Click it like a nine-year-old would.

Four checks before you go further

Run these in order. If any fails, the sentence beside it is what you paste back.

Do this	If it misbehaves, say this
Click a wrong answer on purpose. It should gray out and invite another try.	"When I click a wrong answer, nothing happens. It should gray that choice out and let me try another one."
Then click the right answer. It should turn green and explain why.	"When I get it right, it does not tell me why. Show the explanation under the question."
Check the counter. A question you got wrong first should NOT count as first-try correct.	"The first-try counter is going up even when I guessed wrong first. It should only count questions I got right on my first click."
Answer everything. A summary should appear at the end.	"Nothing happened when I finished the last question. Show a summary with how many I got right on the first try."

The third check is the one that catches people out. It is the most common thing to come back subtly wrong, and it is the thing that makes the activity honest: a student who guesses three times and lands on the right answer has not shown you they know it.

Put your own content in

Now it works, make it yours. You have two ways, and the second one is the one worth learning.

The fast way – ask

PROMPT 2 · YOUR CONTENT

Replace the example questions with these, keeping everything else exactly as it is. For each one, write the explanation so it teaches the idea rather than just restating the answer.

[paste your questions and answers here, in any rough format]

The way worth learning – edit it yourself

Open the file in your text editor. Near the top you will find the question list. Change the words between the quote marks. Save. Refresh the browser. That is the whole skill, and it means you are never waiting on an AI to fix a typo.

Write the explanations as if the student got it wrong. "50, because 47 is past halfway" teaches. "The answer is 50" does not. This is the single biggest difference between an activity that drills and an activity that instructs, and no AI will make that judgment for you — it is the part that is actually your job.

STEP 3, CONTINUED

The part you edit

This is what the question list looks like in the finished file. Four things to notice.

```
{ q: "Round 47 to the nearest ten.",  
  choices: ["40", "50", "47", "500"],  
  answer: 1,  
  why: "47 is past the halfway point (45), so it rounds up to 50." },
```

Part	What it is
q	The question the student reads. Change the words, keep the quote marks.
choices	The buttons, in the order they appear. Four is a habit, not a rule — three or five work.
answer	Counts from zero. The first choice is 0, the second is 1, the third is 2. Here the answer is "50", which sits second, so it is 1.
why	What the student sees after answering. This is where the teaching lives.

Counting from zero is the one thing that trips everybody. If a question marks the wrong choice as correct, this is almost always why. Count the choices starting at zero and check the number matches.

Always test your own key

Click through every question once after you swap the content in. An activity that confidently marks the wrong answer correct is worse than no activity, and it takes ninety seconds to rule out.

Make the feedback teach

A right-or-wrong activity is worth something. A right-or-wrong activity that explains itself is worth ten times as much, and it costs one more prompt.

PROMPT 3 · BETTER WRONG-ANSWER FEEDBACK

For each question, give each *wrong* choice its own short message explaining the likely mistake — not just "try again". Keep it to one sentence, keep it kind, and never say the student is wrong as a person. Show that message when they click that specific choice.

What that turns into

A student who clicks "40" on "round 47" sees:

"Close — 47 is past the halfway mark, so it goes up rather than down."

Rather than: *"Not quite — try again."*

This is the part where being a teacher beats being a programmer. You know which wrong answer means "guessed", which means "rounded the wrong way", and which means "does not know what halfway is." Those are three different students and they need three different sentences. Write them yourself.

Worth knowing

The included file uses one shared try-again message, to keep the code simple enough to read. Prompt 3 is how you upgrade it. Both versions are legitimate — start where you like.

Make it look like your room

PROMPT 4 · APPEARANCE

Change the look: use [your colors] as the main colors, make the question text larger, and put the activity title and a short instruction line at the top. Keep the buttons big enough to tap on a tablet.

PROMPT 5 · ACCESSIBILITY, AND THIS ONE MATTERS

Make sure this works for a student using a keyboard instead of a mouse: every choice should be reachable by pressing Tab and selectable with Enter, with a clearly visible outline on whatever is currently selected. Do not use color alone to show right and wrong — include words too.

Prompt 5 is not optional and it is not decoration. A student using a keyboard, a screen reader, or with any color vision difference will hit a wall on an activity that signals "correct" with green alone. Asking for it costs one prompt. Retrofitting it later costs an afternoon.

How to check it worked

Put the mouse down. Press Tab repeatedly — you should see a clear outline moving between the choices. Press Enter on one — it should select. If either fails, say exactly that and ask again.

Get it out of the chat and into a file

Until you do this, the thing you made lives inside a conversation. This step is what makes it yours.

1. Copy all of the code

Most tools have a copy button on the code block. Use it — selecting by hand usually misses the last line.

2. Paste it into a plain text editor

Notepad, or TextEdit set to plain text. **Not Word.** Word adds invisible formatting that stops the file working, and the failure is baffling when it happens.

3. Save it with .html on the end

Name it something you will recognize: rounding-practice.html. The .html is what tells your computer to open it in a browser. On Windows, set "Save as type" to All Files so it does not quietly become a .txt.

4. Double-click it

It opens in your browser. No internet needed. That is the whole thing — it works the same on a school laptop with the wifi off.

If it opens as a page of code instead of an activity, the file extension did not save. Rename it so it ends in .html and try again. This is the single most common stumble in this whole pack and it is a thirty-second fix.

USING IT

Getting it to students

Four ways, easiest first. All of them keep the file yours.

How	What happens	Worth knowing
On the board	Open it yourself and work through it as a class.	Zero setup. Good for a first run.
Shared drive	Drop it in the class folder students already open.	Uses access your district already granted.
Attach in your LMS	Post it as a file in Classroom, Canvas, Schoology.	Some LMSs preview HTML instead of opening it — tell students to download first.
On the devices	Copy it onto classroom laptops or tablets once.	Works with the wifi off. Genuinely useful when the network is down.

Notice what is not on this list: putting it on the public internet. You do not need to, and not needing to is what keeps this simple. The file works fine sitting on a drive.

One thing to check with your district

Some districts restrict what file types students can open, or block local HTML on managed devices. That is a two-minute question to your tech team and it is worth asking before you plan a lesson around it.

The six things that actually go wrong

In order of how often they happen. Each fix is one sentence you paste back into the chat, or one thing you change yourself.

1. It opens as a wall of code

Cause: the file did not save with .html on the end. **Fix:** rename it so it ends in .html. On Windows you may need to turn on "file name extensions" in File Explorer's View menu to see what it really saved as.

2. Nothing happens when you click

Cause: usually the code was copied incompletely — the last lines got missed. **Fix:** copy it again with the copy button, not by selecting with the mouse.

3. It marks the wrong answer correct

Cause: the answer number counts from zero and it is off by one. **Fix:** count the choices starting at 0 and correct the number. This is the most common error in your *own* edits, not in the AI's work.

4. It looks plain and unstyled

Cause: the styling came as a separate file. **Fix:** "Put the styling inside the same HTML file so I only have one file to save."

Two more, and a rule

5. It asks for a name or a login

Cause: the AI added a feature you did not ask for — it does this sometimes, trying to be helpful. **Fix:** "Remove the name field and any login. This must not collect anything about the student." Say it plainly; it will comply. And notice that this is the Tier 1 rule from page 4 doing its job.

6. It worked, then you changed something, and now it does not

Cause: a missing quote mark or comma in the question list. **Fix:** compare your edit to the pattern on page 10. Every question needs its quote marks closed and a comma between entries. If you cannot spot it, paste the question list into the chat and say "something in this list is broken — find it."

The rule that covers everything not on this list:

Describe what you saw, in ordinary words, and ask for it to be fixed. You do not need the right vocabulary. "The green thing does not appear when I get it right" is a completely serviceable bug report and it will be understood.

When to stop fixing and start over

If three rounds have not fixed it, the conversation has probably tied itself in a knot. Start a fresh chat, paste Prompt 1 again, and add the one thing that went wrong. This is faster than untangling, and it costs you two minutes.

Changing it for anything else

The activity is not about rounding. Rounding was just the example. Here is the same file doing other jobs — each of these is one prompt or one edit away.

Subject	What the questions become
Vocabulary	The word, with four meanings. The explanation gives the word in a sentence.
Reading	A short passage at the top, then comprehension questions about it.
Science	A diagram described in words, then "which part does X?"
History	A source quotation, then questions about who wrote it and why.
Grammar	A sentence with four possible corrections.
Music, art, PE	Terms, signs, rules — anything with a right answer and a reason.

PROMPT 6 · CHANGING THE SUBJECT ENTIRELY

Keep this activity exactly as it works now, but replace the content with [subject] questions for [grade]. Write the explanations so they teach the idea, and keep the reading level appropriate for that age.

The test of whether a topic fits: can a question have one clearly right answer and a reason you could explain in a sentence? If yes, it works. If the answer is "it depends" or "there are several good answers", this is the wrong tool — that is a discussion, not a drill.

What not to build this way

Knowing where this stops is as valuable as knowing how to start. Four honest limits.

Anything that grades for real

This is practice, not assessment. It keeps no record, and it should not. A score that counts belongs in the system your district already uses, where it is backed up, auditable and covered by policy.

Anything holding a student's name

Page 4, Tier 3. The temptation arrives the moment the tool becomes genuinely useful. Keep the roster key somewhere else.

Anything open-ended

Essays, opinions, "explain your thinking" — a click-the-answer activity cannot judge those, and pretending otherwise teaches students that thinking has four options. Use this for the recall that frees your time for the open-ended work.

Anything you would not check yourself

If you would not read every question before handing it out on paper, do not hand it out on a screen either. The medium changed; your responsibility for the content did not.

Tier 2: using real results safely

The activity in this pack collects nothing. But once you have built one tool, you will want one that looks at how students actually did. Here is how to do that without crossing a line.

What makes data de-identified

Not "I removed the last names." De-identified means a person who found this file could not work out which child it describes — even knowing your class. Student numbers, no names, no initials, no "the one with the IEP", no detail that only fits one child.

A worked example

Safe to put in a tool or a prompt:

S1: 8/10 S2: 6/10 S3: 9/10 S4: 4/10
Whole group missed Q3 and Q7 (both regrouping)

Not safe, even though it looks harmless:

Marcus: 4/10 - still struggling since his IEP meeting

Keep the key on paper. A sticky note in your planner that says S1 = Marcus does the whole job, and it never leaves your desk. The tool sees S1 forever.

Tier 3: when to stop and ask

Sooner or later you will want to build something that genuinely needs real student information. That is not a failure of nerve — it is the point at which this stops being a solo project.

You are in Tier 3 the moment any of these is true

- Names, initials or student ID numbers go into the tool.
- It stores anything between sessions — a database, an account, a saved file of results.
- It lives on the public internet where a link would reach it.
- It holds grades that count, attendance, behavior records or anything from an IEP.
- Somebody other than you would need to log in to it.

What to do instead of abandoning the idea

Take it to your district's technology team as a proposal, not a fait accompli. Districts say yes to things reviewed in advance far more often than to things discovered in use. Bring what it does, what data it touches, and where it would live. Peninsula School District in Washington built its own tools this way and reports saving substantial money doing it — with the district involved, which is exactly the difference.

Why the district cares, in one sentence: student records are covered by federal and state law, and the school — not you personally — carries the obligation. A tool they reviewed is a tool they can stand behind.

Practical guidance, not legal advice. Your district's policy, your state's student privacy law, FERPA and COPPA are the authorities. Ask your district before real student information enters anything you build.

The short version

The rest of this pack explains why. This page is what to do. Come back for the reasoning once it is working.

1 • Open the file that came with this pack

practice-activity.html. Double-click it. It opens in your browser and works immediately. You now have a working activity and you have written nothing.

2 • Open it again in a text editor

Right-click > Open with > Notepad or TextEdit. Near the top is a block marked YOUR QUESTIONS GO HERE.

3 • Change the words between the quote marks

Your question, your four choices, your explanation. Remember answer counts from zero. Save the file.

4 • Refresh the browser

Your activity. Ten minutes, no AI needed at all for this route.

Then, when you want a different *kind of activity* rather than different questions — that is when the prompts on page 6 onward earn their keep. The AI is for changing what the thing does. Editing the list is for changing what it asks.

BEFORE YOU BUILD

Plan it first – five minutes with a pen

The teachers who get a genuinely useful tool are the ones who decided what it was for before they opened the chat. Fill this in.

What skill is this practicing?

One skill. Not a unit. "Rounding to the nearest ten", not "place value".

What are the three wrong answers your students actually give?

These become your distractors. Invented wrong answers teach nothing; the ones your class really produces teach a great deal.

For each one – what is the misconception behind it?

This is what you write in the feedback. It is the part no AI can do for you, and the part that makes the activity worth more than a worksheet.

Ten questions is plenty. A student who does ten with real feedback learns more than one who clicks through forty. Resist the urge to generate a hundred just because you can.

A SECOND EXAMPLE

The same file, doing a different job

To show this is not really about rounding — here is the identical structure holding a vocabulary question, written the way this pack argues for.

```
{ q: "In the sentence 'the path was treacherous after the rain',  
  what does TREACHEROUS mean?",  
  choices: ["Very muddy", "Dangerous and unstable", "Long and winding", "Steep"],  
  answer: 1,  
  why: "Treacherous means dangerous, especially in a hidden way - the rain made the  
        path unsafe, not just messy." },
```

Why each wrong choice is there

Choice	The student who picks it
Very muddy	Took the context clue (rain) but not the word. Very common, and worth a specific message.
Long and winding	Guessed from "path" rather than from the word at all.
Steep	Knows it is something about difficulty, has not pinned down which.

That table is the actual work. Writing four plausible choices is easy. Knowing which wrong answer means which misunderstanding is teaching, and it is why your version of this activity will beat a generated one.

What else the same loop builds

You now know a method, not a product. The loop on page 5 is identical for all of these — describe, look, say what is off, repeat.

Build	Tier	What changes
Flashcard drill	1	One thing at a time, flip to check, shuffle each run.
Sorting activity	1	Drag items into categories. Ask for click-to-select if dragging proves fiddly on tablets.
Timed fluency practice	1	A countdown and a running count. Good for math facts.
Class hub page	1	Today's agenda, links, due dates. You edit it; nobody logs in.
Parent explainer	1	What we are learning and how to help at home, in one page you can send.
Score pattern viewer	2	Paste de-identified scores, see which questions the group missed. Read page 19 first.

Notice the Tier column. Five of those six need nobody's permission. The sixth touches real results and so it follows the Tier 2 rules — student numbers, no names, key on paper. Knowing which column you are in is the whole discipline.

The honest summary of what you have learned

Not how to code. How to describe a thing precisely enough that a machine can build it, and how to tell whether what came back is right. Those are teaching skills. You already had them; this pack pointed them at a new object.

Every prompt, in one place

Copy from here rather than retyping. Square brackets mean "put your own thing here".

1 • FIRST BUILD

Build me a self-checking practice activity as one single HTML file I can save and open in a browser. Students see questions with four clickable choices. A wrong click greys that choice out and invites another try; a right click turns green, locks the others and explains why it is right. A counter shows how many were right on the FIRST try. Show a summary at the end with a start-over button. Everything in one file, no internet, no login, no student name, nothing saved. Put my questions in a clearly marked list at the top with comments explaining how to add one. Big text and big buttons for tablets.

2 • YOUR CONTENT

Replace the example questions with these, keeping everything else exactly as it is. Write each explanation so it teaches the idea rather than restating the answer. [your questions]

3 • BETTER WRONG-ANSWER FEEDBACK

Give each wrong choice its own one-sentence message explaining the likely mistake, shown when that specific choice is clicked. Keep it kind.

4 • APPEARANCE

Use [your colors] as the main colors, make the question text larger, and add a title and instruction line at the top. Keep buttons big enough to tap.

5 • ACCESSIBILITY

Make every choice reachable with Tab and selectable with Enter, with a clearly visible outline on the current one. Do not use color alone to show right and wrong — include words too.

6 • CHANGE THE SUBJECT

Keep this working exactly as it does, but replace the content with [subject] questions for [grade], at an appropriate reading level.

Three more worth keeping

7 · WHEN IT WILL NOT BEHAVE

Here is what I did and what happened: [what you clicked]. I expected [what should have happened]. Fix it and give me the whole file again.

8 · SHUFFLE THE QUESTIONS

Put the questions in a different random order each time the page is opened, so students sitting together do not see the same sequence. Keep each question's choices with it.

9 · A PRINTABLE VERSION

Add a print stylesheet so that printing this page gives a clean worksheet: questions and choices only, no buttons, no colors, no score panel, with room to write.

Prompt 9 is the one teachers ask for second. The same question bank becomes the screen activity and the paper version, and no one has to retype anything. It is also the answer to "what do I do when the laptops do not turn up."

A habit worth forming

Keep your own prompts in a document as you find ones that work. Within a term you will have a personal library that fits how you teach, and building the next tool becomes a matter of pasting rather than thinking.

THE FILE

The finished activity, in full

The complete file is included with this pack as `practice-activity.html` — you do not need to type any of this. It is printed so you can see what you are working with, and so the pack still makes sense on paper.

This first part is the only bit you edit. Everything after it is machinery.

```
<!-- =====
YOUR QUESTIONS GO HERE. This is the only part you need to change.

Each question has:
  q      the question the student reads
  choices the options they can click
  answer the position of the correct one, counting from 0
         (first choice = 0, second = 1, third = 2, fourth = 3)
  why    what the student sees after they answer, right or wrong

Add or delete questions freely. Two is fine. Thirty is fine.
===== -->
<script>
const ACTIVITY_TITLE = "Rounding to the Nearest Ten";
const ACTIVITY_NOTE  = "Click the answer you think is right. You can try again.";

const QUESTIONS = [
  { q: "Round 47 to the nearest ten.",
    choices: ["40", "50", "47", "500"],
    answer: 1,
    why: "47 is past the halfway point (45), so it rounds up to 50." },

  { q: "Round 23 to the nearest ten.",
    choices: ["30", "20", "25", "200"],
    answer: 1,
    why: "23 has not reached the halfway point (25), so it rounds down to 20." },

  { q: "Round 85 to the nearest ten.",
    choices: ["80", "90", "85", "100"],
    answer: 1,
    why: "85 is exactly halfway. The rule we use is to round up, so 85 becomes 90." },

  { q: "Round 61 to the nearest ten.",
    choices: ["70", "60", "61", "600"],
    answer: 1,
    why: "61 is only one past 60, well short of halfway, so it rounds down to 60." },

  { q: "Round 8 to the nearest ten.",
    choices: ["0", "10", "8", "80"],
    answer: 1,
    why: "8 is past halfway between 0 and 10, so it rounds up to 10." }
];
</script>
```

How this pack is built

What we are claiming

That the prompts in this pack, used with a current AI chat assistant, produce a working self-checking activity. Every prompt here was run and the resulting activity was tested before publication — including clicking wrong answers on purpose, checking the first-try counter behaves honestly, and confirming the answer key is right.

What we are not claiming

We are not claiming this replaces your district's learning management system, and we would advise against trying. We are not claiming it is an assessment tool. We are not claiming any particular learning outcome. And this is not legal advice about student data — your district's policy, your state's law, FERPA and COPPA are the authorities.

An honest limitation

AI tools change. A prompt that produces a clean single file today may produce something slightly different in six months. That is why this pack teaches the *loop* on page 5 rather than only the prompts — the loop survives the tools changing, and the included working file does not depend on any AI at all.

Sources

Peninsula School District's use of AI-built internal tools, and the associated cost saving, is as reported in 2026 coverage of the district's technology work. Student privacy guidance follows the Family Educational Rights and Privacy Act (FERPA) and the Children's Online Privacy Protection Act (COPPA); the Future of Privacy Forum's *Educator's Guide to Student Privacy* is a readable starting point. Neither this pack nor its authors are affiliated with any AI company or any named district.

Built and audited by two certified teachers. Every prompt run, every step followed start to finish, and the included file tested in a browser before this pack shipped.

If this was useful

Leaving feedback earns you credit

TPT gives you credit toward future purchases every time you review something you have downloaded. Go to **My Purchases**, find this pack, and leave a rating and a comment. It costs a minute and it comes back to you as money off the next thing you buy.

It also tells us what to build next, which is genuinely how the decisions get made here.

If something did not work

Please tell us through TPT before leaving a rating — message us and we will fix it and re-upload the same day, and you keep the updated file free. A prompt that stopped behaving is something we want to know about, not something to live with.

What you can do now that you could not this morning:

Build a practice activity for any topic you teach, in about ten minutes, that checks itself and explains its answers — without a subscription, a login, or anyone's permission. The rounding example was never the point. The loop on page 5 was.

Built and audited by two certified teachers

Bright Scholar · AI-Ready School